Tyre Shop Service Booking and Management System for

C M Kurian & Company

A project report submitted to the

DEPARTMENT OF COMPUTER APPLICATIONS

in partial fulfillment of the requirements
for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

BY

NIRANJAN S

REG NO.RA2432241010224

UNDER THE GUIDANCE
OF
Dr. S. VANITHA M.Sc.,M.Phil.,Ph.D.,NET

DEPARTMENT OF COMPUTER APPLICATIONS

FACULTY OF SCIENCE AND HUMANITIES

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

Kattankulathur – 603 203

Chennai, Tamil Nadu

April – 2026

BONAFIDE CERTIFICATE

This is to certify that the project report titled **"TYRE SHOP SERVICE BOOKING MANAGEMENT SYSTEM"** is a Bonafide work carried out by **NIRANJAN S (RA24322410102224)** under my supervision for the award of the Degree of Master of Science in Applied Data Science. To my knowledge the work reported herein is the original work done by this student.

Dr. S. Vanitha
Assistant Professor,
Department of Computer Applications
(GUIDE)

Dr. P. J. Arul Leena Rose
Professor & Head
Department of Computer Applications
(HEAD OF THE DEPARTMENT)

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION OF ASSOCIATION OF RESEARCH PROJECT WITH SUSTAINABLE DEVELOPMENT GOALS

This is to certify that the research project entitled "**TYRE SHOP SERVICE BOOKING MANAGEMENT SYSTEM"** carried out by **Mr Niranjan S** under the supervision of **Dr. S.Vanitha** in partial fulfilment of the requirement for the award of post-graduation program has been significantly or potentially associated with SDG Goal No **09 (NINE)** titled **INDUSTRY, INNOVATION AND INFRASTRUCTURE**

This study has clearly shown the extent to which its goals and objectives have been met in terms of filling the research gaps, identifying needs, resolving problems, and developing innovative solutions locally for achieving the above-mentioned SDG on a National and/or on an international level.

SIGNATURE OF THE STUDENT **GUIDE**

HEAD OF THE DEPARTMENT

ACKNOWLEDGEMENT

With profound gratitude to the ALMIGHTY, I take this chance to thank people who helped me to complete this project.

I take this as a right opportunity to say THANKS to my parents who are there to stand with me always with the words “YOU CAN”.

I wish to express my sincere gratitude to **Dr.T.R. Paarivendhar**, Chancellor, SRM Institute of Science & Technology who gave us the platform to establish me to reach greater heights.

I am thankful to **Prof. A.Vinay Kumar**, Pro Vice-Chancellor (SBL) and **Dr. A.Duraisamy**, Dean, Faculty of Science and Humanities, SRM Institute of Science & Technology for their unwavering support throughout my project.

I earnestly thank **Dr. S. Albert Antony Raj,** Professor and Deputy Dean, College of Sciences, Faculty of Science and Humanities who always encourage me to do novel things.

I express my sincere thanks to **Dr. P. J. Arul Leena Rose,** Professor and Head, Department of Computer Applications, Faculty of Science and Humanities for her valuable guidance and support to execute all incline in learning.

It is my delight to thank my project guide **Dr. S.Vanitha,** Assistant Professor, Department of Computer Applications for her help, support, encouragement, suggestions, and guidance throughout the development phases of the project.

I convey my gratitude to all the faculty members of the department who extended their support through valuable comments and suggestions during the reviews.

A great note of gratitude to friends and people who are known and unknown to me who helped in carrying out this project work a successful one.

NIRANJAN S

C.M. KURIAN & COMPANY

TYRES WHEEL ALIGNMENT

16-March-2026

This is to certify that Mr. Niranjan S, MCA (Computer Applications) student of SRM Institute of Science and Technology, has successfully completed the project titled "Tyre Shop Service Booking and Management System for C M Kurian & Company."

The project involved the design and development of a web-based system to manage customer service bookings, service records, and related shop operations for C M Kurian & Company.

The project was carried out from January 2026 to March 2026.

We acknowledge his efforts and dedication toward the successful completion of the project and wish him success in his future endeavors

For C M Kurian and Company

For C.M. KURIAN & COMPANY

Partner

C.M. KURIAN & COMPANY Chalakuzhy Bhavan S.H. Mount M.C. Road Ph: 2568765 KOTTAYAM

Lino C Kurian

Managing Partner

H.O: CHALAKUZHY BHAVAN, S.H. MOUNT, M.C. ROAD, **KOTTAYAM** Ph: **0481-256 8765, 230 1765**
Branch: CHALAKUZHY BUILDING, K.K. ROAD, ILLIVALAVU, **MANARCADU**, KOTTAYAM Ph: **8281371785**

Copyleaks

Scan details

Scan Time	Total Pages	Total Words
March 19th, 2026 at 13:34 UTC	1	213

Analysis Report

Plagiarism Detection Report

New Scans 1:34 PM

Plagiarism Detection

0%

Plagiarism types	Text coverage	Words
Identical	0%	0
Minor Changes	0%	0
Paraphrased	0%	0
Excluded		
Omitted Words		0

AI Detection

N/A

	Text coverage
AI Text	
Human Text	
Excluded	
Omitted Words	

Plagiarism

Results (0)

*Results may not appear because the feature has been disabled.

Private Cloud Hub 0	**Shared Data Hub** 0	**Filtered / Excluded** 0
Internet Sources 0	**AI Source Match** N/A	**Current Batch** 0

Plagiarism types	Text coverage	Words
Identical	0%	0
Minor Changes	0%	0
Paraphrased	0%	0
Excluded		
Omitted Words		0

About Plagiarism Detection

Our AI-powered plagiarism scans offer three layers of text similarity detection: Identical, Minor Changes, and Paraphrased. Based on your scan settings we also provide insight on how much of the text you are not scanning for plagiarism (Omitted words).

Identical

One to one exact word matches. Learn more

Minor Changes

Words that hold nearly the same meaning but have a change to their form (e.g. "large" becomes "largely"). Learn more

Paraphrased

Different words that hold the same meaning that replace the original content (e.g. 'large' becomes 'big') Learn more

Omitted Words

The portion of text that is not being scanned for plagiarism based on the scan settings. (e.g. the 'Ignore quotations' setting is enabled and the document is 20% quotations making the omitted words percentage 20%) Learn more

Copyleaks Shared Data Hub

Our Shared Data Hub is a collection of millions of user-submitted documents that you can utilize as a scan resource and choose whether or not you would like to submit the file you are scanning into the Shared Data Hub. Learn more

Filtered and Excluded Results

The report will generate a complete list of results. There is always the option to exclude specific results that are not relevant. Note, by unchecking certain results, the similarity percentage may change. Learn more

Current Batch Results

These are the results displayed from the collection, or batch, of files uploaded for a scan at the same time. Learn more

TABLE OF CONTENTS

1. INTRODUCTION 1

1.1 STATEMENT OF OBJECTIVES……1

1.2 SCOPE OF THE PROJECT……2

2. SOFTWARE REQUIREMENT ANALYSIS 3

2.1 HARDWARE SPECIFICATION 3

2.2 SOFTWARE SPECIFICATION 3

3. SYSTEM ANALYSIS 4

3.1 REQUIREMENT SPECIFICATION 4

3.2 CHARACTERISTICS OF EXISTING SYSTEM 5

3.3 FEASIBILITY STUDY 5

3.4 SOFTWARE REQUIREMENT SPECIFICATION 7

4. SYSTEM DESIGN 9

4.1 SYSTEM ARCHITECTURE 9

4.2 USE CASE DIAGRAM 12

4.3 ACTIVITY DIAGRAM 13

4.4 CLASS DIAGRAM 14

4.5 DATA FLOW DIAGRAM 15

4.6 ER DIAGRAM 17

5. SYSTEM IMPLEMENTATION 18

5.1 MODULE DESCRIPTION 18

5.2 SYSTEM WORKFLOW 19

5.3 TECHNOLOGY IMPLEMENTATION……...20

5.4 DATA SECURITY IMPLEMENTATION……21

5.5 REPORT GENERATION……...21

6. TESTING 22

6.1 TEST CASES ..... …..22

6.2 TESTING PROCEDURES……23

6.3 TEST REPORT ..... .25

7. RESULT AND CONCLUSION 30

7.1 RESULT 26

7.2 CONCLUSION 26

7.3 FUTURE ENHANCEMENTS……..27

8. APPENDICES 29

8.1 SCREEN SHOTS 29

8.2 SAMPLE CODING 37

9. REFERENCES 52

BOOK REFERENCES 52

WEB REFERENCES 52

ABSTRACT

The growth of digital technologies has encouraged many service industries to adopt online systems that improve efficiency and customer convenience. However, several small and medium tyre service centers still depend on manual booking methods such as phone calls or in-person scheduling. These traditional approaches often lead to problems such as booking conflicts, poor record keeping, and difficulty in accessing service information. The Tyre Shop Service Booking Management System developed for C M Kurian & Company provides a web-based solution that simplifies the process of scheduling tyre services. Through the system, customers can create accounts, log in, explore available services, and submit booking requests online. During the booking process, users can choose required services, describe vehicle issues, select preferred service dates, and monitor the status of their requests. The platform also includes an administrative panel that enables staff to review booking requests, manage customer records, track completed services, and analyze business performance. Additional capabilities such as automated email notifications, invoice generation, and downloadable business reports improve transparency and help the shop maintain organized service records. All information related to users and service bookings is stored in a MySQL database, ensuring reliable data management. By digitizing the booking workflow, the system minimizes manual effort and provides a structured platform for managing tyre service operations efficiently

LIST OF FIGURES

S NO	FIGURE NO	FIGURE NAME	PAGE NO
1	4.1.1	System Architecture	11
2	4.2.1	Use Case Diagram	12
3	4.3.1	Activity Diagram	13
4	4.4.1	Class Diagram	14
5	4.5.1	Level 0 DFD	15
6	4.6.1	Level 1 DFD	16
7	4.6.2	ER Diagram	17

LIST OF SCREENSHOTS

S NO	PHOTO NO	PHOTO NAME	PAGE NO
1	8.1.1	Home Screen	29
2	8.1.2	Login Page	29
3	8.1.3	Sign Up Page	30
4	8.1.4	User Dashboard	30
5	8.1.5	Admin Dashboard	31
6	8.1.6	Service Booking	31
7	8.1.7	Service Cart	32
8	8.1.8	Booking Details	32
9	8.1.9	Service History	33
10	8.1.10	Revenue Report	33
11	8.1.11	Analytics Page	34
12	8.1.12	Invoice	34
13	8.1.13	Excel Data	35
14	8.1.14	Analytics Image	35
15	8.1.15	Confirmation Mail	36
16	8.1.16	Cancellation Mail	36

1. INTRODUCTION

The increasing use of digital technologies has encouraged organizations to adopt online platforms to enhance the quality and efficiency of their services. Despite this trend, many tyre service centers still depend on traditional booking approaches such as telephone requests or in-person appointments. These manual practices often create problems including poor scheduling management, difficulty in organizing service records, and reduced convenience for customers. To address these issues, the Tyre Shop Service Booking Management System provides a web-based platform that simplifies the process of requesting tyre-related services. Customers can access the system online, explore the range of services offered by the shop, choose the services they require, and submit booking requests through an interactive and easy-to-use interface. The system also includes a dedicated administrative panel that enables shop staff to efficiently handle service operations. Through this panel, administrators can review customer bookings, verify customer details, schedule service appointments, and maintain records of completed services. In addition, the application offers analytical features such as revenue summaries and service performance reports, which support business owners in evaluating operational performance and making informed management decisions. This system was developed for C M Kurian & Company Tyres with the goal of improving the efficiency of service booking procedures, maintaining organized digital records of services, and strengthening customer interaction through a modern web-based solution.

1.1 STATEMENT OF OBJECTIVES

The primary objective of this project is to develop a web-based tyre service booking and management system that improves service efficiency and customer experience:

- To create a centralized platform that stores and manages customer details, service requests, and booking information digitally.
- To allow customers to explore available tyre services and schedule service appointments online through a simple interface.

- To enable administrators to manage bookings, confirm or cancel service appointments, and maintain accurate service records.
- To generate invoices and maintain organized records of completed services along with financial information.
- To provide analytical insights and booking statistics that help business owners evaluate service performance and demand.

1.2 SCOPE OF THE PROJECT

The scope of this project includes:

- Developing a web-based platform that allows customers to register, log in, and request tyre services through an online booking system.
- Providing a centralized system to store and manage customer information, service bookings, and service history records.
- Enabling administrators to review booking requests, confirm or cancel appointments, and maintain organized service records.
- Generating invoices and financial reports for completed services to help the business track revenue and maintain proper documentation.
- Providing business analytics and statistical insights that allow administrators to monitor service demand and evaluate shop performance.
- Implementing a system that improves communication between customers and the tyre shop by providing booking status updates and service notifications.

2. SOFTWARE REQUIREMENT ANALYSIS

2.1 HARDWARE SPECIFICATION

- Operating System: Windows 10 or higher.
- Processor: Intel Core i5 or higher or equivalent AMD processor.
- Ram: Minimum 8GB (suggested: 16GB or higher)
- Storage: The Installation of necessary software programs and the storage of datasets and outputs requires at least 100GB of free disk space.

2.2 SOFTWARE SPECIFICATION

- Operating System: Windows (Compatible with n8n and PostgreSQL)
- Programming Languages: HTML, CSS, React JS, Node JS, Express JS
- Programming IDE: Visual Studio Code
- Database Management System: MySQL

3. SYSTEM ANALYSIS

3.1 REQUIREMENT SPECIFICATION

The system requires a well-organized and reliable structure to manage tyre service bookings, customer information, and business operations efficiently. The main requirements of the system include:

- **Customer Information Management:** Collecting and storing customer details such as name, contact number, email address, and account credentials through the user registration module. This information is used to manage bookings and maintain service records.
- **Service Booking Processing:** Allowing customers to browse available tyre services, select the required services, and submit booking requests with details such as preferred service date, branch location, and vehicle problem description.
- **Booking Management:** Providing administrators with the ability to review booking requests, verify customer information, confirm or cancel service appointments, and update booking statuses within the system.
- **Service and Invoice Management:** Recording completed service jobs and generating invoices that include service details, pricing information, and customer information for proper financial documentation.
- **Business Analytics and Reporting:** Analyzing booking data to generate revenue reports, service statistics, and graphical insights that help administrators evaluate business performance.
- **Secure Data Storage:** Maintaining all user accounts, service bookings, and financial records in a MySQL database to ensure reliable data storage and secure management of system information.

3.2 CHARACTERISTICS OF EXISTING SYSTEM

The current tyre service booking process used in many small and medium tyre service centers has several limitations and operational challenges:

- **Manual Booking Process:** Service appointments are commonly recorded through phone calls or direct customer visits, and the details are often written in notebooks or simple registers. This manual approach increases the chances of scheduling conflicts and record errors.
- **Poor Record Management:** Customer information and service details are not stored in a structured digital system, making it difficult to maintain accurate records of previous services and customer history.
- **Limited Customer Convenience:** Customers must contact the service center directly or visit the shop to request services, which can be inconvenient and time-consuming.
- **Lack of Business Insights:** Traditional systems do not provide tools to analyze service demand, track revenue, or evaluate business performance, making it difficult for owners to make informed decisions.
- **No Automated Communication:** In manual systems, customers are not automatically informed about booking confirmations, cancellations, or service updates, which can lead to communication gaps between the shop and customers.

The proposed system addresses these limitations by introducing a digital service booking platform that improves service management, maintains organized records, and enhances communication between customers and administrators.

3.3 FEASIBILITY STUDY

The feasibility study evaluates whether the proposed system can be successfully developed and implemented based on different factors.

- **Technical Feasibility:**
    - The system is developed using modern web technologies such as HTML, CSS, JavaScript, React.js, Node.js with Express.js, and MySQL for database

management.

- These technologies support the development of a scalable web application capable of handling user registrations, service bookings, and administrative operations efficiently.

- The application can be developed and executed on standard computer systems using development tools such as Visual Studio Code and commonly available web browsers.

- **Economic Feasibility:**
  - The project mainly uses widely available and cost-effective technologies, which helps reduce development and maintenance expenses.
  - Digital management of bookings and service records minimizes manual work and reduces operational inefficiencies.
  - The system supports better financial tracking through automated invoice generation and revenue reporting.
- **Operational Feasibility:**
  - The system provides a user-friendly interface that allows customers to book services easily and enables administrators to manage bookings and service records efficiently.
  - Automated features such as booking management, invoice generation, and reporting help reduce manual workload.
  - The integrated modules ensure a smooth workflow from service selection and booking submission to service completion and record management.
- **Legal and Ethical Feasibility:**
  - The system maintains secure storage of customer information and booking records within the database.
  - Access to administrative features is restricted to authorized users to ensure proper management and protection of system data.

- Proper data handling practices help maintain privacy and responsible use of customer information within the system.

3.4 SOFTWARE REQUIREMENT SPECIFICATION

The software requirement specification defines the functional and non-functional requirements:

Functional Requirements:

- The system allows customers to create accounts, log in securely, and manage their personal information within the platform.
- The system enables users to browse available tyre services, view service descriptions and prices, and select services according to their requirements.
- The system allows customers to submit service booking requests by providing details such as phone number, preferred service date, branch location, and vehicle problem description.
- The system enables administrators to review booking requests, confirm or cancel service appointments, and update booking status within the system.
- The system records completed service jobs and automatically generates invoices that customers can download.
- The system stores user accounts, service bookings, and financial records in a MySQL database for reliable data management.
- The system provides reporting and analytics features that allow administrators to monitor revenue, service demand, and overall business performance.

Non-Functional Requirements:

- System Scalability: The system should be able to handle an increasing number of users, service bookings, and stored records without performance issues.
- Performance: The system should process booking requests, retrieve service information, and update records efficiently.
- Security: The system should protect user information through secure authentication and restricted access to administrative features.

- Reliability: The application should operate consistently and maintain accurate records without system failures or data loss.
- Usability: The system should provide a clear and user-friendly interface so that customers and administrators can easily navigate and use the platform.
- Compatibility: The application should function smoothly on standard computer systems and web browsers using technologies such as React.js, Node.js, Express.js, and MySQL.

4. SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

Overview

The system architecture of the Tyre Shop Service Booking Management System is divided into two major components:

Customer Service Booking System – This part of the system focuses on customer account management, browsing available tyre services, and submitting service booking requests through an online platform.

Service Management and Business Analysis System – This component handles the management of service bookings, monitoring of completed services, invoice generation, and analysis of business performance.

Customer Service Booking System

Purpose

To allow customers to access the platform, view available tyre services, and schedule service appointments conveniently through an online interface.

Components

User Account Module

Manages customer registration, login authentication, and user profile information within the system.

Service Collection Module

Displays the list of tyre services offered by the shop along with service descriptions and pricing details.

Service Booking Module

Allows customers to select services, enter booking details such as phone number, preferred service date, branch location, and vehicle problem description, and submit booking requests.

Service Management and Business Analysis System

Purpose

To help administrators manage service bookings, maintain service records, generate invoices, and analyze business performance.

Components

Booking Management Module

Enables administrators to review incoming booking requests, confirm or cancel appointments, and update booking status.

Job History Module

Maintains records of completed service jobs including customer details, services performed, and billing information.

Business Analytics Module

Analyzes service data to generate revenue statistics, service demand trends, and performance reports that assist business owners in decision-making.

Database (MySQL)

The MySQL database stores all system data including user accounts, service booking details, service records, and financial information. It ensures reliable storage and retrieval of information required for system operations.

Data Flow

- Customers create accounts and log in to the system through the user interface.
- Users browse available tyre services and submit service booking requests with the required details.
- Booking requests are stored in the database and sent to the administrator for review.
- Administrators manage bookings by confirming, completing, or cancelling service appointments.
- After service completion, the system records job details, generates invoices, and updates business analytics data.

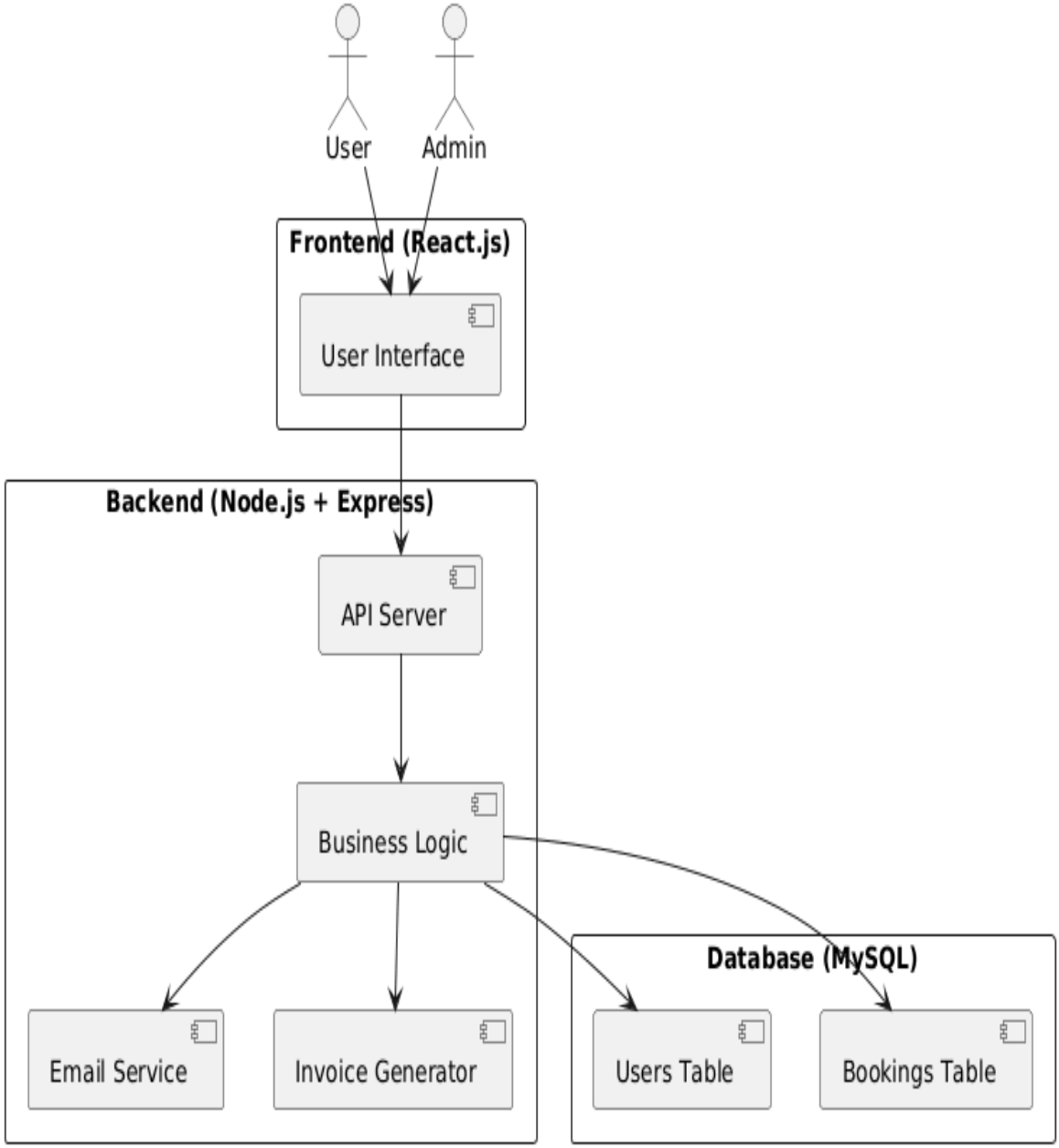

Figure 4.1.1 (SYSTEM ARCHITECTURE)

4.2 USE CASE DIAGRAM

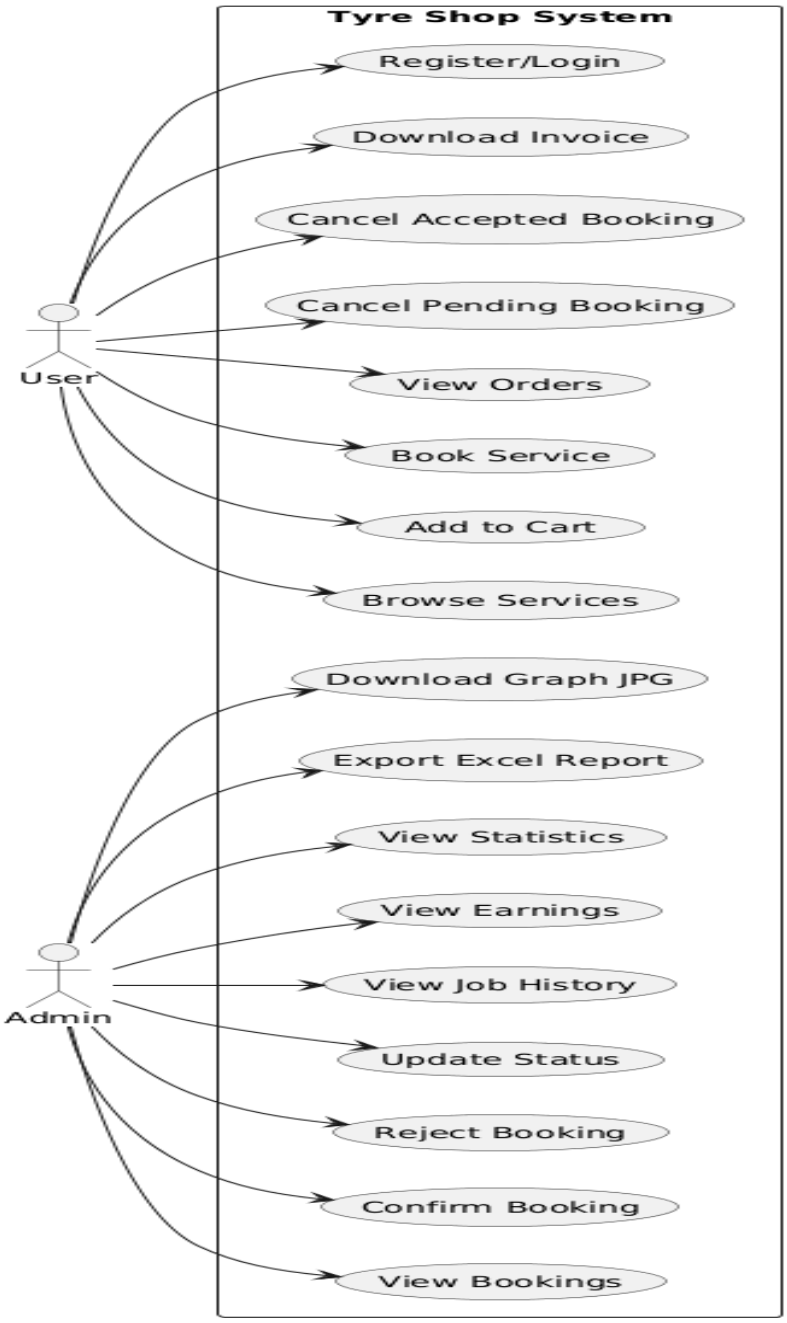

Figure 4.2.1

4.3 ACTIVITY DIAGRAM

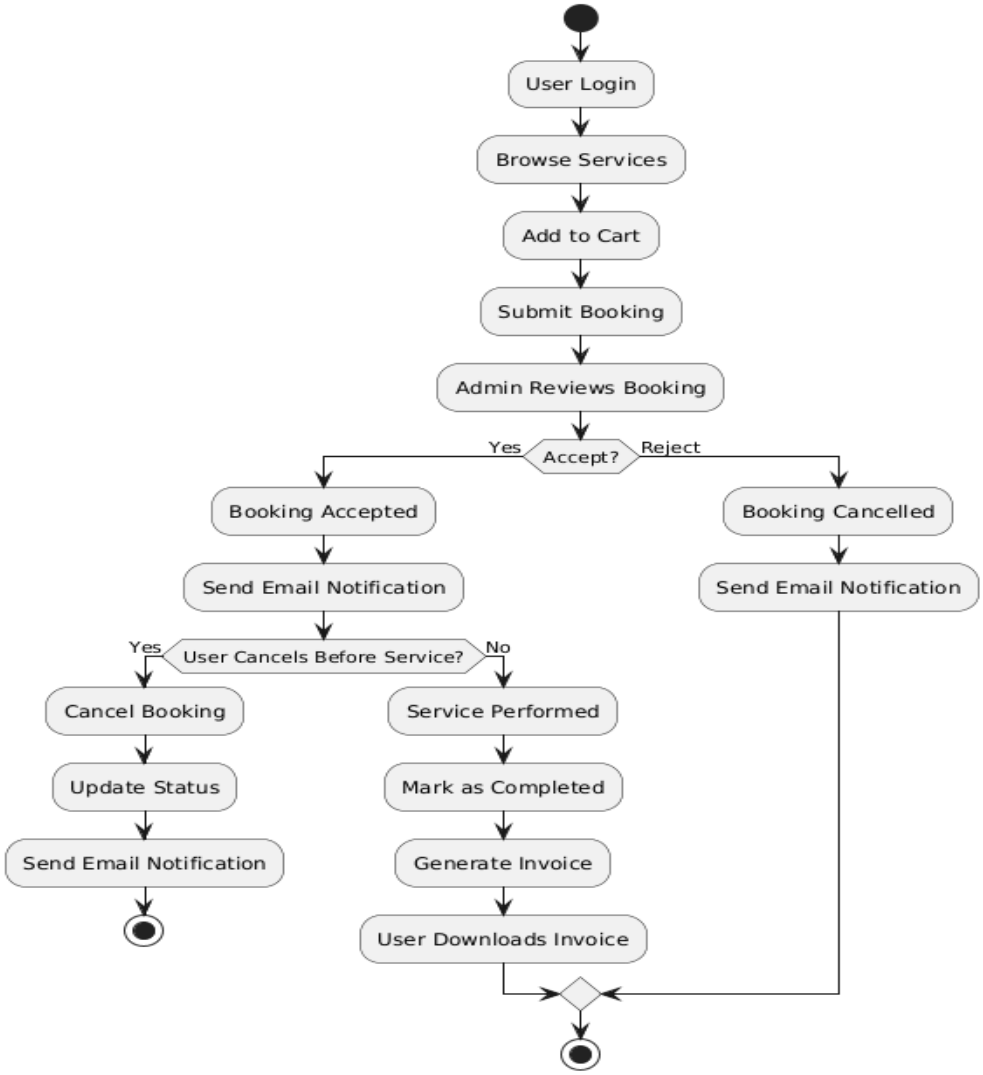

Figure 4.3.1

4.4 CLASS DIAGRAM

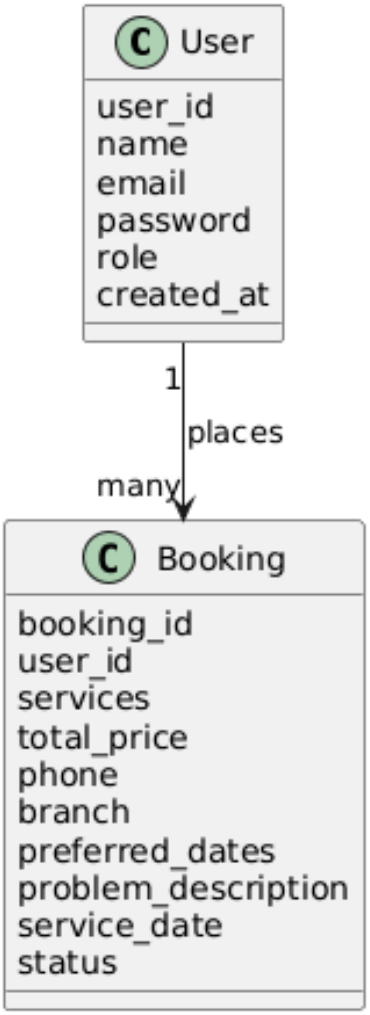

Figure 4.4.1

4.5 DATA FLOW DIAGRAM

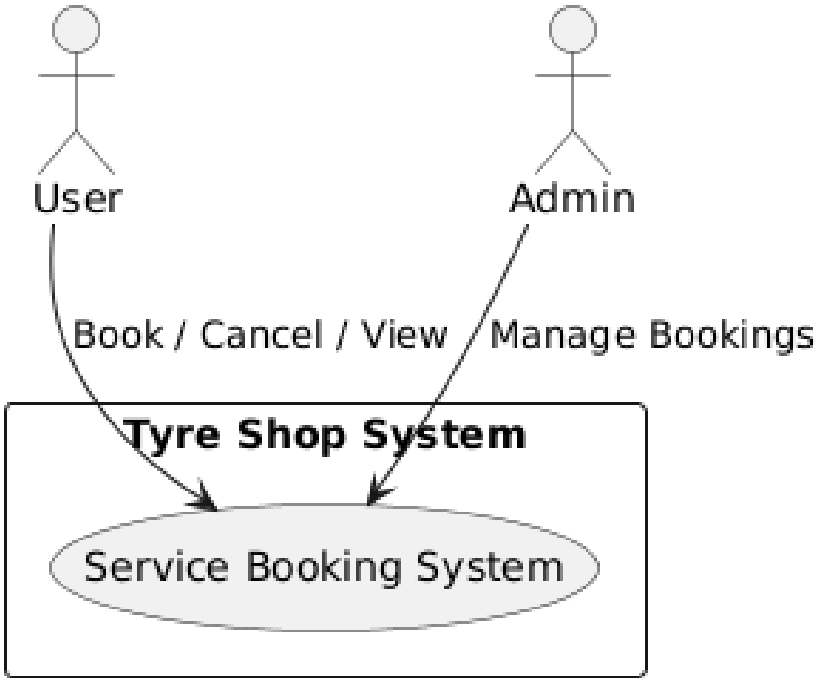

Figure 4.5.1 (Level 0 DFD)

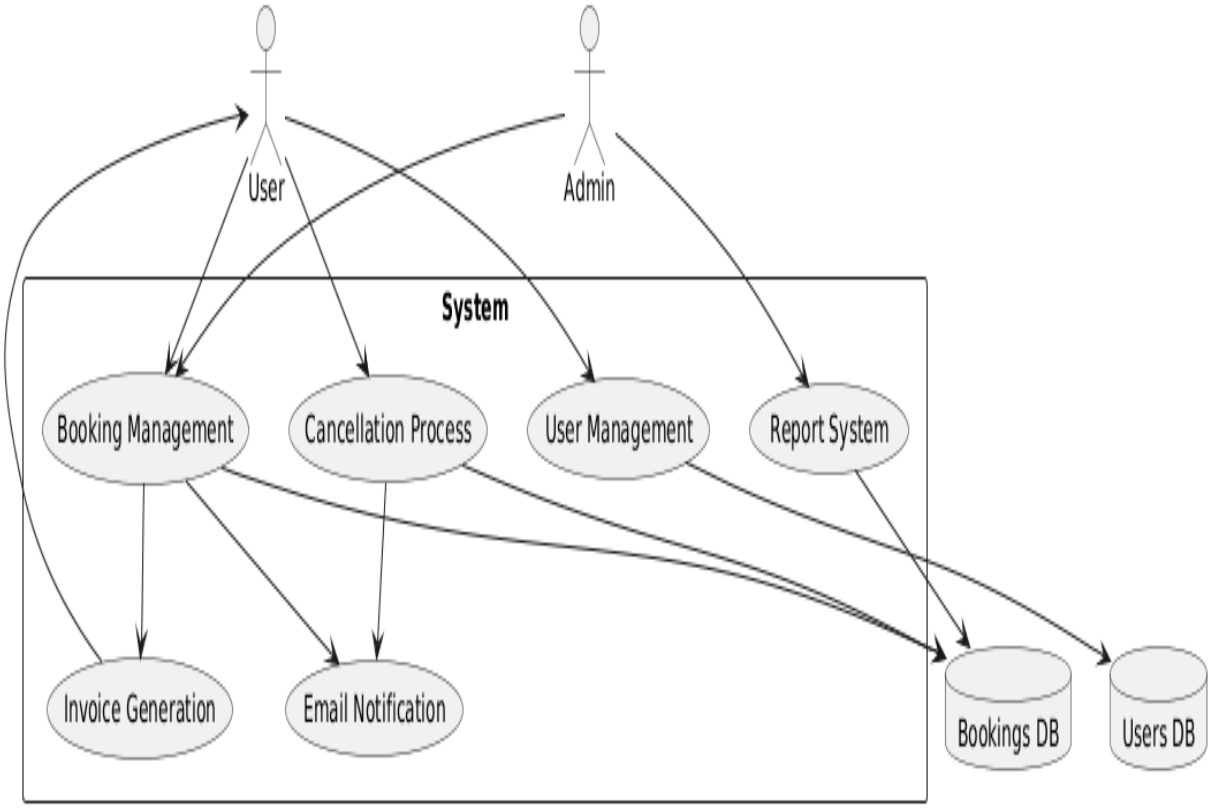

Figure 4.5.2 (LEVEL 1 DFD)

4.6 ER DIAGRAM

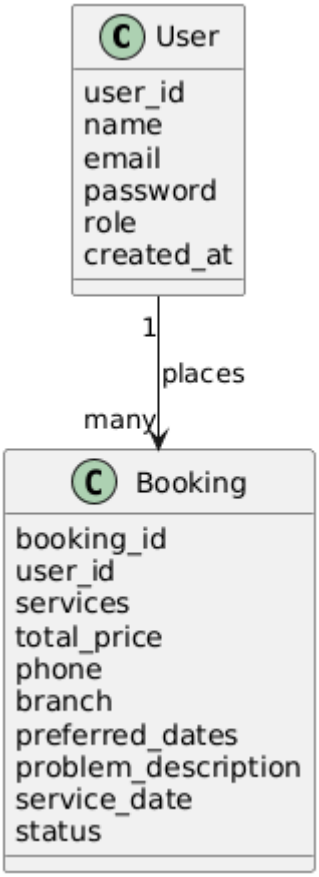

Figure 4.6.1

5. SYSTEM IMPLEMENTATION

5.1 MODULE DESCRIPTION

1. User Management Module

The User Management module handles customer registration, login, and account management within the system. Through this module, customers can create new accounts by providing details such as name, email address, and password. The system verifies login credentials and allows registered users to securely access the platform. This module also manages user profiles and ensures that customer information is stored safely in the database.

2. Service Collection Module

The Service Collection module displays the list of tyre services offered by the shop. Customers can browse different services such as wheel alignment, wheel balancing, tyre replacement, puncture repair, and other maintenance services. Each service includes a brief description and price information. This module helps users understand the available services and choose the appropriate option for their vehicle.

3. Service Booking Module

The Service Booking module allows customers to request tyre services through the online platform. After selecting the required services, users provide booking details such as phone number, preferred service date, branch location, and vehicle problem description. The system processes this information and stores the booking request in the database. This module simplifies the appointment scheduling process for customers.

4. Booking Management Module

The Booking Management module is used by administrators to monitor and control all incoming booking requests. Administrators can review customer booking details, verify the requested services, and confirm or cancel service appointments. The module also allows administrators to update booking statuses such as accepted, completed, or cancelled. This ensures proper organization of service operations.

5. Reporting and Analytics Module

The Reporting and Analytics module provides insights into the business performance of the tyre shop. It analyzes booking data to generate reports related to service demand, completed jobs, and total revenue. Administrators can view statistics and graphical reports that help them evaluate operational performance and make informed business decisions. This module also supports the generation of financial reports for record keeping.

5.2 SYSTEM WORKFLOW

Initially, customers create an account in the system by providing basic information such as name, email address, and contact details. After successful registration, users can log in to the platform and access the available tyre services. The system displays different services such as wheel alignment, wheel balancing, tyre replacement, puncture repair, and other maintenance options.

Once logged in, customers can browse the list of services and select the required options. The selected services are added to a service cart where users can review the chosen services and the total cost. After confirming the cart, the user submits a service booking request by entering details such as phone number, preferred service date, branch location, and vehicle problem description.

The booking request is stored in the database and sent to the administrator for review. The administrator accesses the booking management module to verify the request and either confirm or cancel the service appointment. Once confirmed, the booking status is updated in the system.

After the service is completed at the tyre shop, the system records the job details and generates an invoice for the customer. The invoice can be downloaded by the user through the booking history section.

Finally, the system updates business reports and analytics, allowing administrators to monitor completed services, track revenue, and analyze overall business performance.

5.3 TECHNOLOGY IMPLEMENTATION

The proposed system is developed using modern web technologies to provide an efficient, reliable, and scalable platform for managing tyre service bookings. The frontend of the application is built using HTML, CSS, and JavaScript, along with React.js to create an interactive and responsive user interface. These technologies help in designing web pages such as the user dashboard, service collection page, booking interface, and administrative panels, allowing users to interact with the system easily.

The backend of the system is implemented using Node.js with the Express.js framework. The backend handles server-side operations such as processing booking requests, managing user authentication, updating booking statuses, and generating invoices. It also manages communication between the frontend interface and the database, ensuring that user actions are processed efficiently.

For data storage and management, the system uses a MySQL database. The database stores information related to registered users, service bookings, completed service records, and financial details. This structured database design allows the system to retrieve, update, and manage information effectively.

The system also integrates additional tools and libraries to improve functionality. jsPDF is used to generate downloadable invoices for completed services, while Nodemailer is used to send automated email notifications to customers regarding booking confirmations, cancellations, and service updates.

By combining these technologies, the system provides a reliable digital platform that simplifies tyre service booking, improves record management, and supports efficient business

operations.

5.4 DATA SECURITY IMPLEMENTATION

Since the system manages customer information and service booking records, appropriate security measures are implemented to protect system data and prevent unauthorized access. The application uses secure authentication mechanisms to ensure that only registered users can access their accounts. User login credentials are stored securely in the database, and access to administrative functions is restricted to authorized administrators.

The system also applies secure communication between the frontend and backend to ensure that user requests and booking information are transmitted safely. Proper validation is performed on user inputs to prevent invalid or malicious data from entering the system.

In addition, the database is protected by controlled access permissions, ensuring that sensitive information such as customer details, booking records, and financial data can only be accessed by authorized system components.

These security practices help maintain the confidentiality, integrity, and reliability of system data while allowing customers and administrators to use the platform safely.

5.5 REPORT GENERATION

The system includes a reporting feature that summarizes service bookings and financial data. After services are completed, the system generates reports containing details such as completed jobs, customer bookings, and total revenue.

These reports help administrators review service activity, identify frequently requested services, and monitor overall business performance. The system also allows reports and analytics charts to be exported for documentation and record-keeping purposes.

6. TESTING

6.1 TEST CASES

The testing process includes the following test cases to ensure system reliability and accuracy:

Test Case ID	Description	Expected Output
TC-01	User registers in the system by entering name, email, and password	User account created and stored successfully in the database
TC-02	User logs into the system using registered credentials	User successfully logs into the system
TC-03	User browses available tyre services	List of available services displayed successfully
TC-04	User selects services and adds them to the service cart	Selected services added to the cart with correct price calculation
TC-05	User submits a service booking request with phone number, service date, and problem description	Booking request stored successfully in the database
TC-06	Administrator reviews booking request	Booking details displayed correctly in admin panel
TC-07	Administrator confirms or cancels the booking	Booking status updated successfully in the system
TC-08	System generates invoice after service completion	Invoice generated successfully and available for download
TC-09	System stores user and booking data securely in the database	Revenue statistics and service reports displayed correctly

TC-10	System stores user and booking data securely in the database	Data stored successfully in MySQL database

6.2 PROCEDURES

Integration Testing

- Integration testing is conducted to ensure that all system modules work together correctly.
- The interaction between modules such as User Registration, Service Booking, Booking Management, and Reporting is verified to ensure smooth data flow.
- This testing confirms that the combined system functions properly when all components are connected and operating together.

Functional Testing

- Functional testing is performed to verify that the system carries out all required operations according to the specified requirements.
- Features such as user registration, login, browsing services, submitting booking requests, and managing bookings are tested.
- This testing ensures that each function produces the expected output for different user inputs.

Performance Testing

- Performance testing evaluates how efficiently the system operates under normal usage conditions.
- The system's response time, booking request processing, and data retrieval from the database are examined.
- This testing ensures that the application maintains stable performance when handling multiple users and service bookings.

Reliability Testing

- Reliability testing ensures that the system operates consistently without failures during regular usage.
- Various scenarios such as booking submissions, status updates, and invoice generation are tested to confirm system stability.
- Error handling is also verified to ensure the system manages unexpected inputs or issues without crashing.

Usability Testing

- Usability testing evaluates how easy it is for users to interact with the system interface.
- The user interface is examined to ensure customers can easily navigate features such as service browsing, booking requests, and viewing booking history.
- Feedback from users is considered to improve the overall user experience and simplify system navigation.

Compatibility Testing

- Compatibility testing verifies that the system works correctly across different web browsers, devices, and operating environments.
- The application interface and system functions are tested to ensure proper operation on commonly used browsers.
- This testing ensures that users can access the system reliably regardless of the platform being used.

6.3 TEST REPORT

Introduction

The purpose of this test report is to provide an overview of the testing activities conducted for the Tyre Shop Service Booking Management System. The testing process ensures that the system operates correctly, efficiently, and reliably according to the defined system requirements.

Test Objective

The main objective of the testing process is to verify that all system functionalities operate as expected. It also aims to evaluate the performance of different modules and identify any errors or issues that may affect the system's operation.

Test Scope

The testing scope includes different modules of the system such as Administration, Assessment, Therapy, Evaluation, and Analytics. It also includes testing of patient data processing, treatment evaluation, prediction analysis, and report generation.

Test Environment

Testing was conducted in a controlled environment using the developed web application running on a local server. The testing process was carried out using common web browsers such as Google Chrome and Mozilla Firefox on a Windows operating system. The development environment included React.js for the frontend, Node.js with Express.js for backend processing, and MySQL for database management.

Test Result

The testing results were satisfactory and confirmed that the system performs the required operations successfully. The modules correctly handle user registrations, process service bookings, manage booking records, and generate invoices and business reports efficiently.

7. RESULT AND CONCLUSION

7.1 RESULT

The Tyre Shop Service Booking Management System provides an integrated platform for managing tyre service requests and business operations in a digital environment. The system successfully combines customer registration, service browsing, booking management, invoice generation, and business analytics within a single web application. The user module allows customers to create accounts, log in to the system, and browse the list of tyre services offered by the shop. Customers can select the required services, provide necessary booking details, and submit service requests through the online interface. These booking requests are stored in the database and can be accessed by administrators for further processing. The administrative module allows shop staff to review incoming bookings, verify customer information, and confirm or cancel service appointments. Once a service is completed, the system records the job details and generates an invoice that customers can download for reference. The system also includes business analytics features that analyze service records and generate reports on completed jobs and revenue statistics. These reports help administrators understand service demand and monitor the financial performance of the tyre shop. Overall, the system demonstrates the ability to manage service bookings efficiently, maintain organized digital records, and provide useful insights that support better business management.

7.2 CONCLUSION

The Tyre Shop Service Booking Management System was successfully developed to improve the efficiency of service booking and management in tyre service centers. The system integrates multiple modules including user registration, service booking, booking management, and business reporting to create a structured digital platform for handling service operations. By replacing traditional manual booking methods with an online system, the application improves service organization and provides greater convenience for

customers. The centralized database ensures that customer information and service records are stored securely and can be accessed when required. The administrative features allow shop staff to manage bookings effectively and maintain proper records of completed services. In addition, the system generates invoices and business reports that assist shop owners in monitoring revenue and evaluating business performance. The user-friendly interface allows customers and administrators to interact with the system easily while maintaining reliable data management. Overall, the system demonstrates how a web-based platform can improve service management, enhance customer interaction, and support efficient operation of tyre service businesses.

7.3 FUTURE ENHANCEMENTS

The system is designed with flexibility and scalability, allowing additional features and improvements to be implemented in future versions. Some possible enhancements are listed below:

1. Mobile Application Support

A mobile application can be developed to allow customers to book tyre services, track booking status, and receive notifications directly from their smartphones.

2. Online Payment Integration

Future versions of the system can include online payment options, allowing customers to pay for services through digital payment gateways during the booking process.

3. Real-Time Service Status Tracking

The system can be enhanced to allow customers to track the real-time status of their service requests, such as booking confirmation, service in progress, or service completion.

4. **Advanced Business Analytics**

Additional analytical tools can be integrated to provide deeper insights into service demand, customer preferences, and revenue trends, helping shop owners make better business decisions.

5. **Multi-Branch Management Support**

The system can be expanded to support multiple tyre shop branches, allowing administrators to manage bookings, services, and reports for different locations within a single platform.

6. **Automated Customer Notifications**

Future improvements can include advanced notification systems using SMS or mobile alerts to inform customers about booking confirmations, service reminders, and completion updates.

7. **Graphical Dashboard for Business Insights**

A visual dashboard can be added to display business statistics, service demand patterns, and revenue performance using charts and graphical reports.

8. **Cloud-Based Deployment**

Deploying the system on cloud platforms can improve system scalability, data storage capacity, and accessibility from different locations.

9. **Customer Feedback and Rating System**

A feedback module can be added where customers can rate services and provide reviews, helping the tyre shop improve service quality and customer satisfaction.

8. APPENDICES

8.1 SCREEN SHOTS

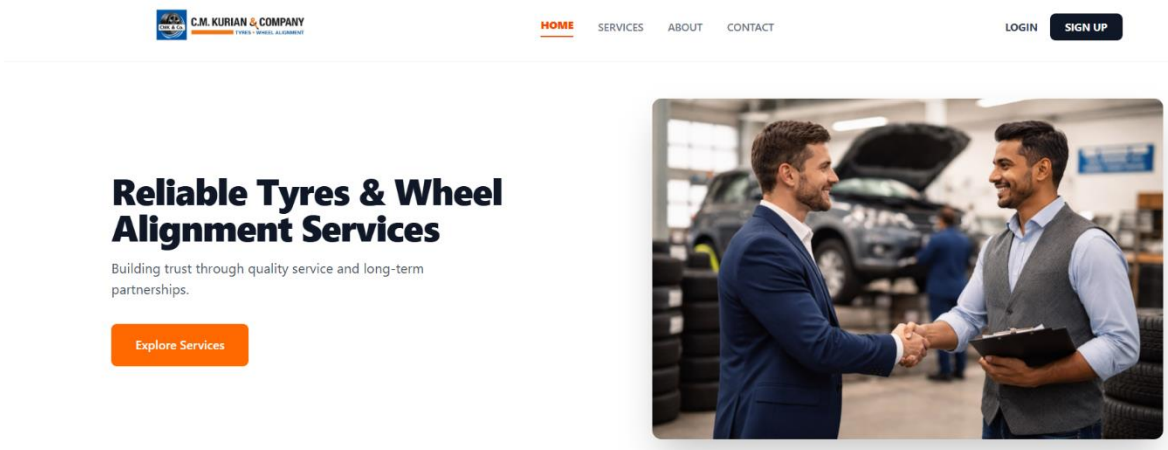

Figure 8.1.1

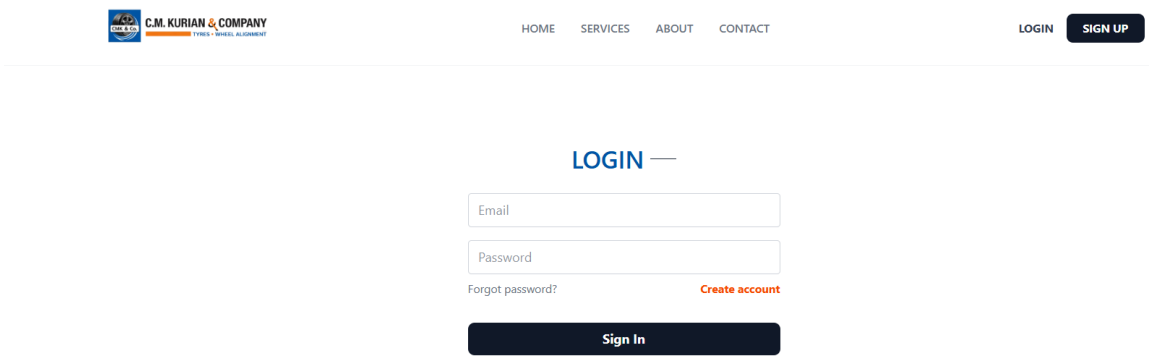

Figure 8.1.2

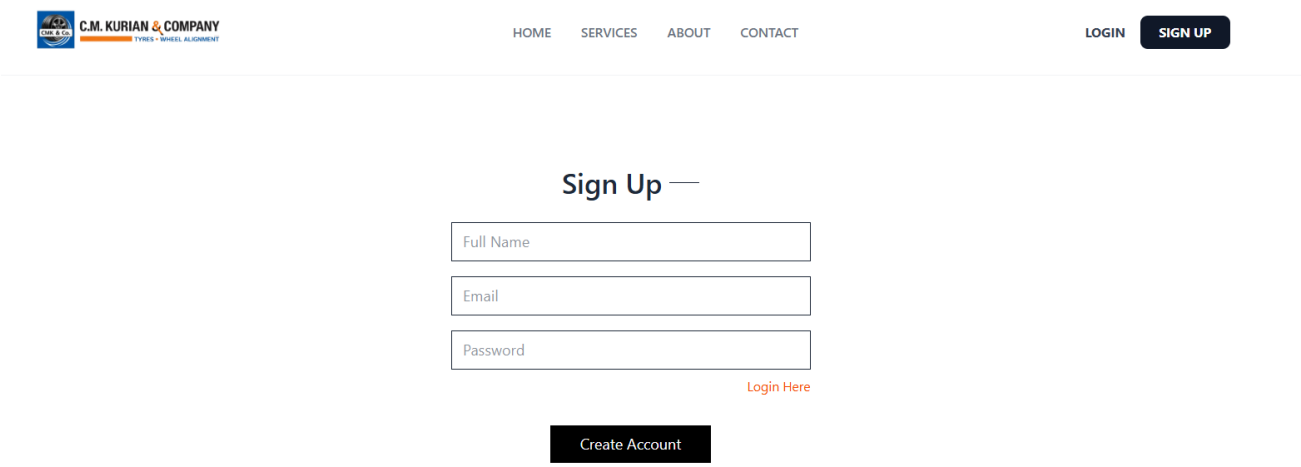

Figure 8.1.3

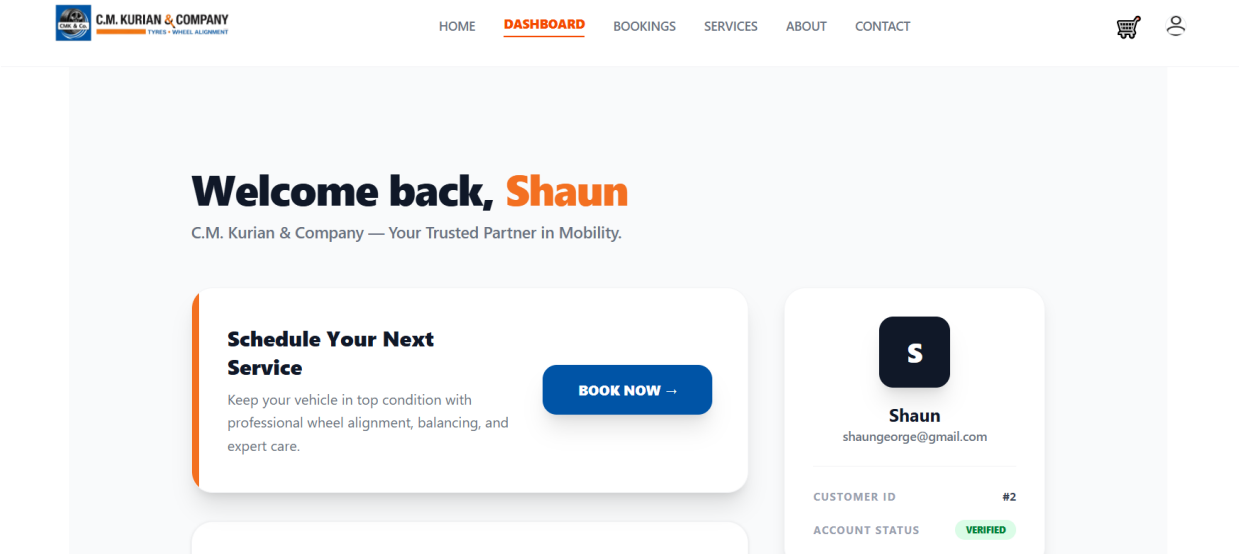

Figure 8.1.4

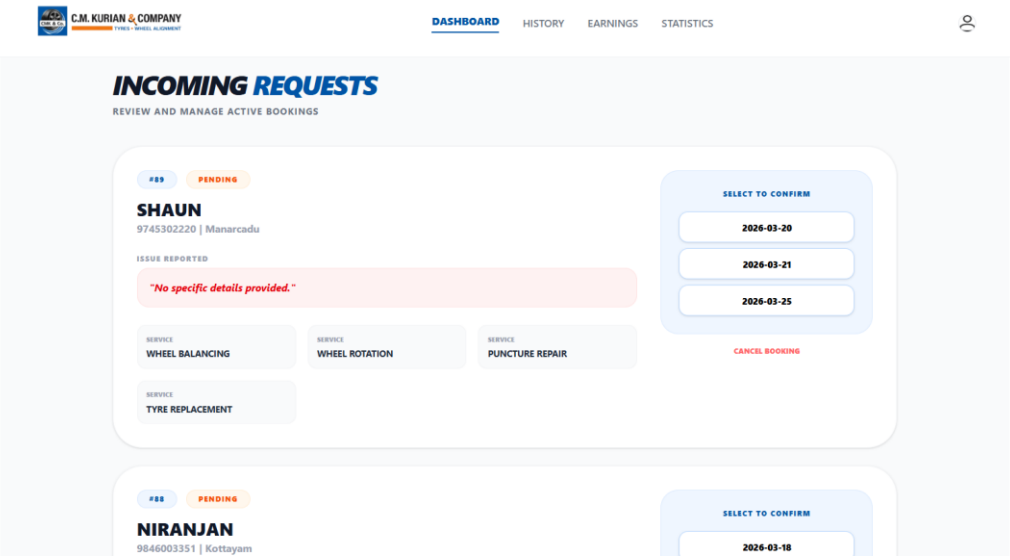

Figure 8.1.5

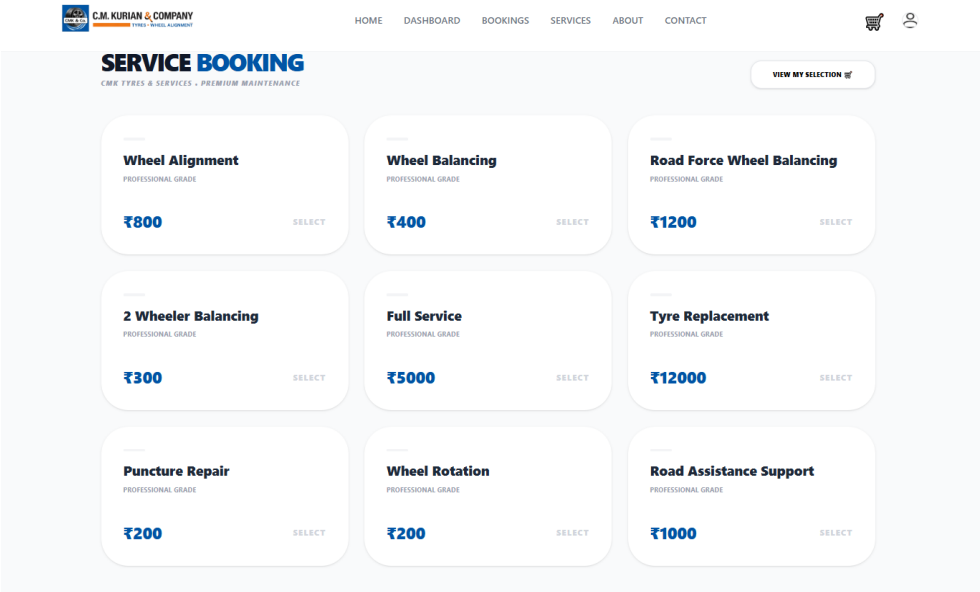

Figure 8.1.6

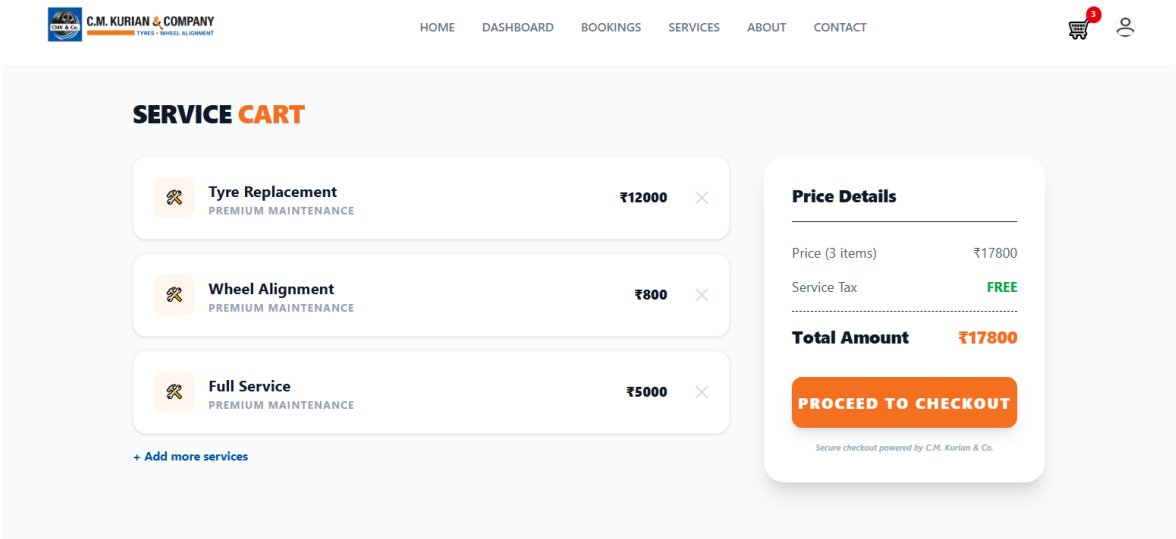

Figure 8.1.7

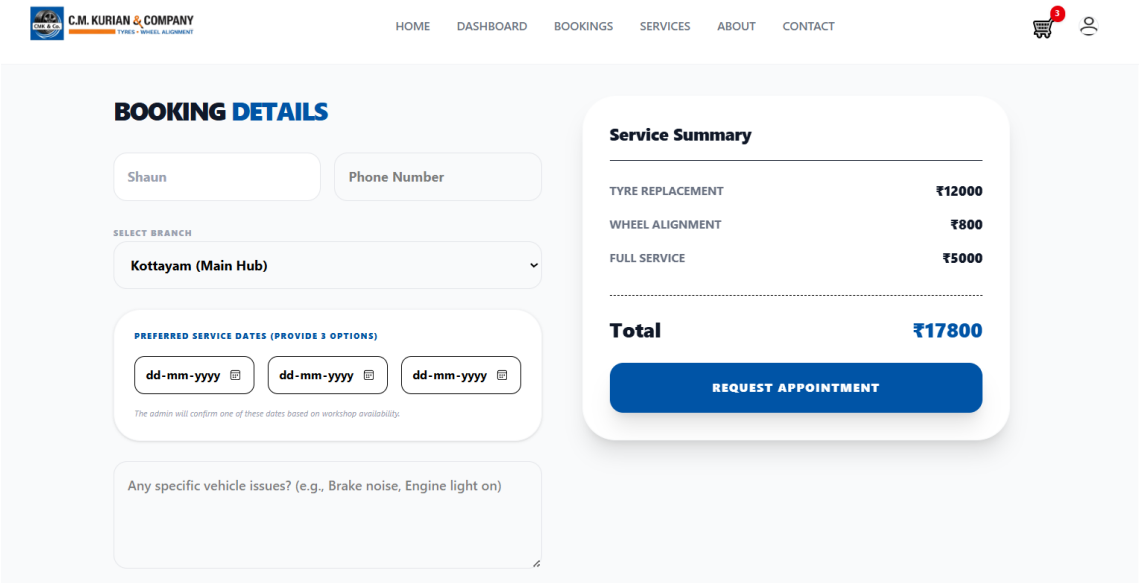

Figure 8.1.8

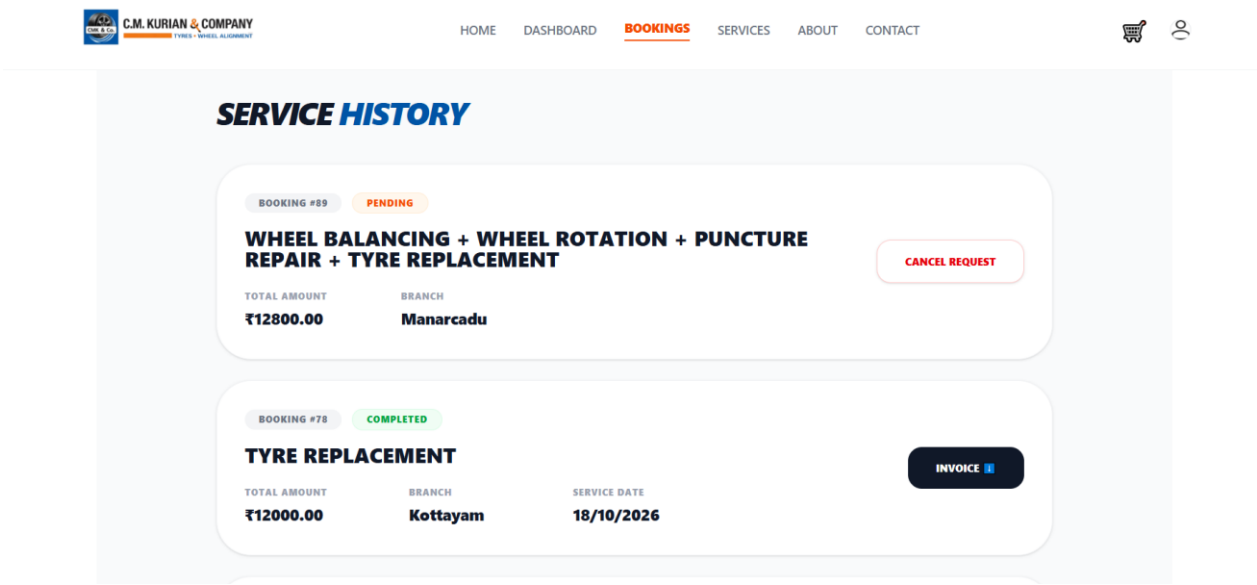

Figure 8.1.9

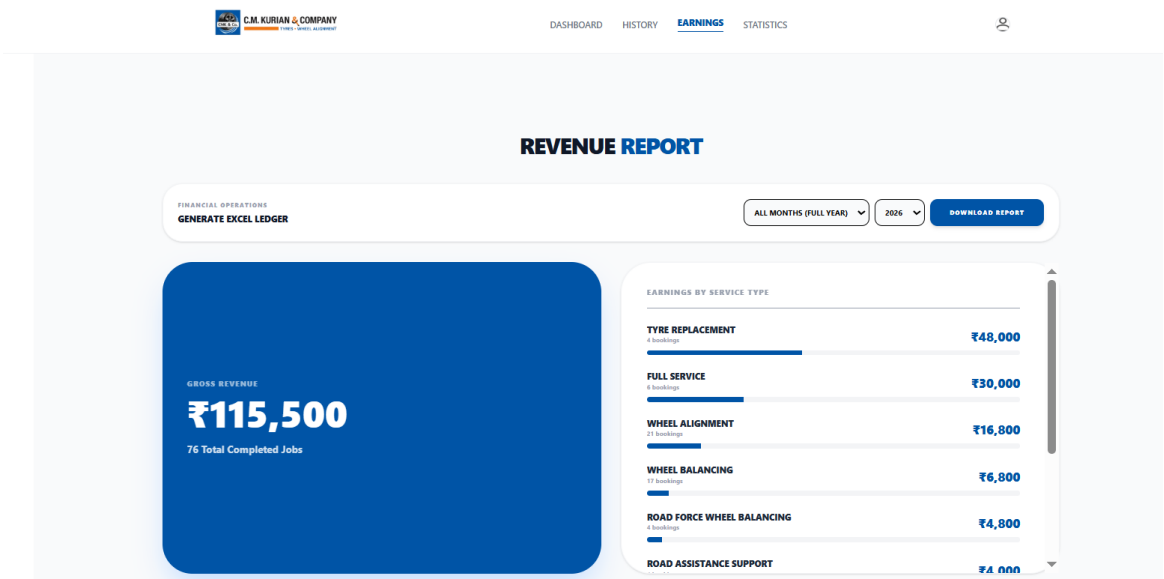

Figure 8.1.10

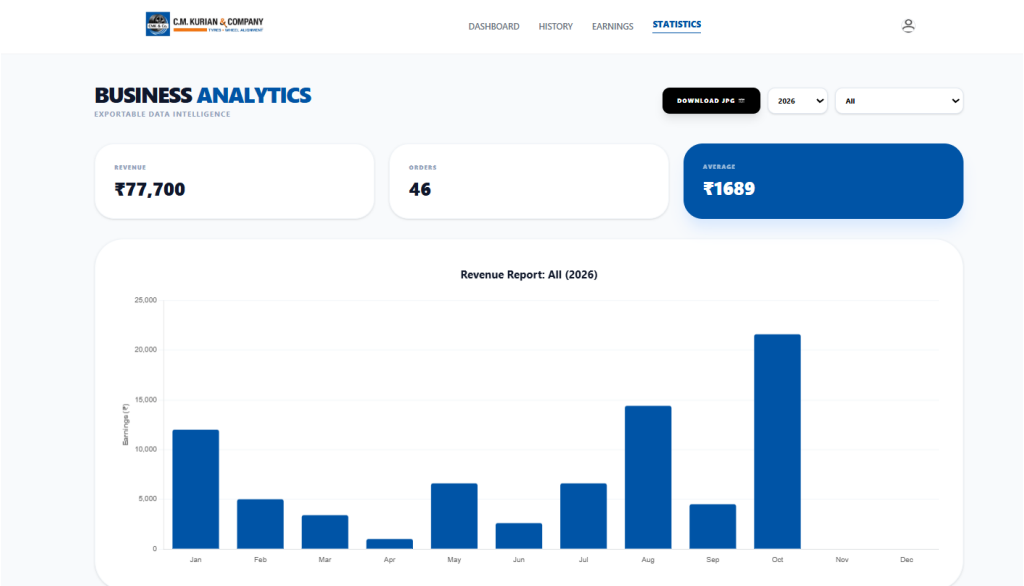

Figure 8.1.11

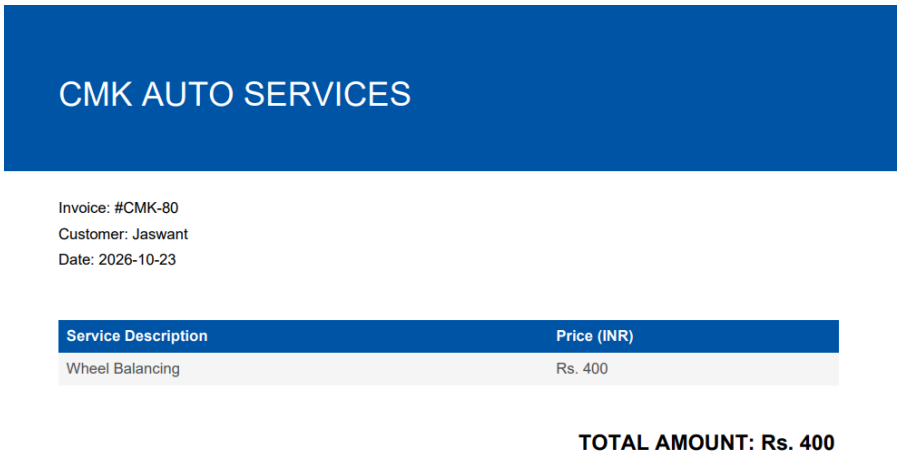

Figure 8.1.12

	A	B	C	D	E
1	Date	Customer Name	Phone	Services	Total Revenue (INR)
2	2026-10-29	Batman	9846020020	Road Assistance Support	1000
3	2026-10-26	Koonari	9846020019	Puncture Repair	200
4	2026-10-23	Jaswant	9846020018	Wheel Balancing	400
5	2026-10-20	Niyas	9846020017	Wheel Alignment	800
6	2026-10-17	Shaun	9846020016	Tyre Replacement	12000
7	2026-10-10	Koonari	9846020014	Wheel Alignment	800
8	2026-10-07	Jaswant	9846020013	Wheel Rotation	200
9	2026-10-04	Niyas	9846020012	Road Force Wheel Balancing	1200
10	2026-10-01	Shaun	9846020011	Full Service	5000
11	2026-09-25	Batman	9846020010	Puncture Repair	200
12	2026-09-22	Koonari	9846020009	Wheel Balancing	400
13	2026-09-19	Jaswant	9846020008	Wheel Alignment	800
14	2026-09-17	Niyas	9846020007	2 Wheeler Balancing	300
15	2026-09-14	Shaun	9846020006	Wheel Balancing, Wheel Alignment	1200
16	2026-09-08	Koonari	9846020004	Wheel Rotation	200
17	2026-09-06	Jaswant	9846020003	Puncture Repair	200
18	2026-09-03	Niyas	9846020002	Wheel Balancing	400
19	2026-09-01	Shaun	9846020001	Wheel Alignment	800
20	2026-08-15	Batman	9846010040	Road Assistance Support	1000
21	2026-08-11	Koonari	9846010039	Puncture Repair	200
22	2026-08-08	Jaswant	9846010038	Wheel Balancing	400
23	2026-08-05	Niyas	9846010037	Wheel Alignment	800
24	2026-08-02	Shaun	9846010036	Tyre Replacement	12000

Figure 8.1.13

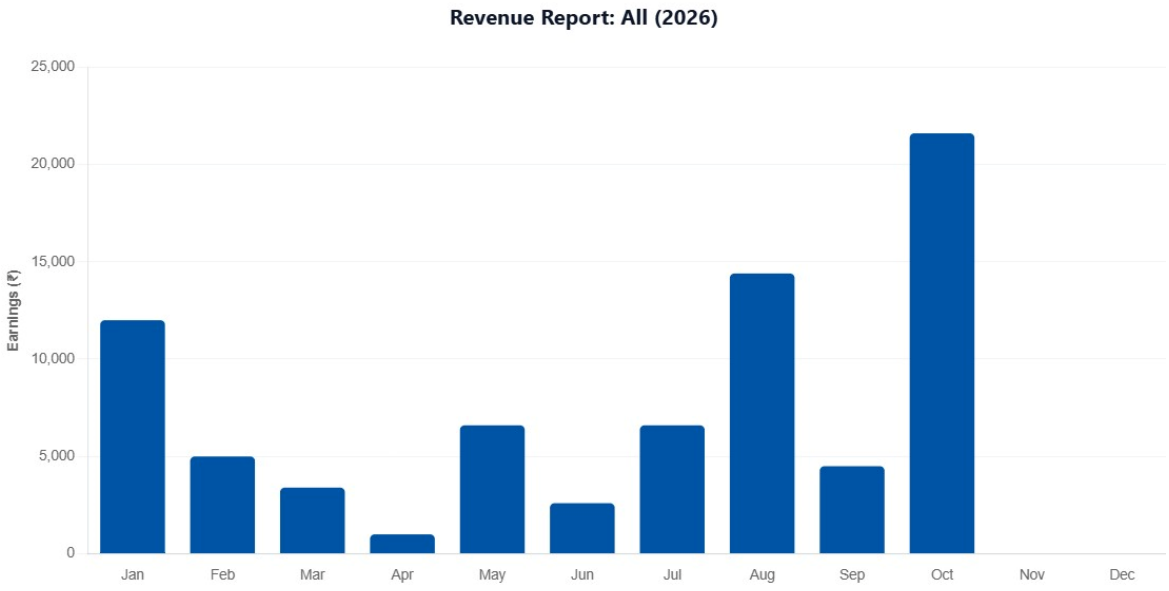

Figure 8.1.14

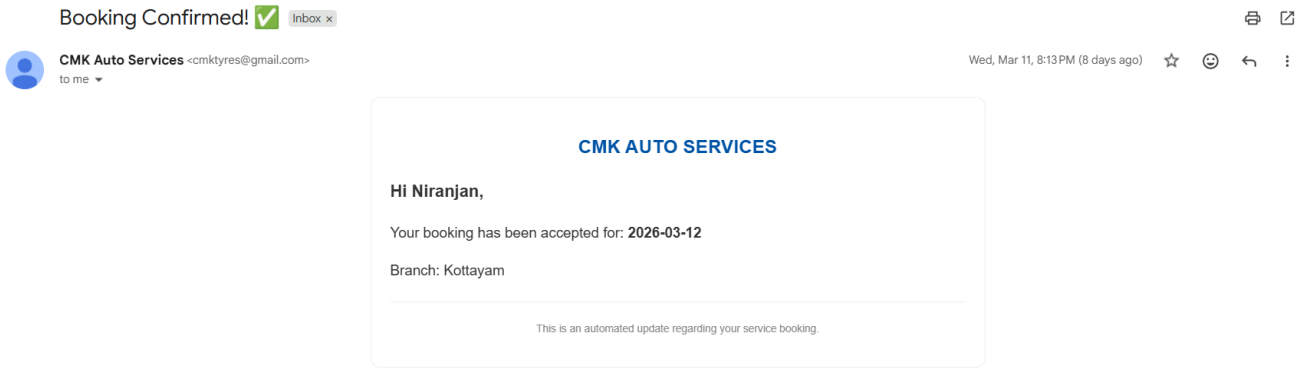

Figure 8.1.15

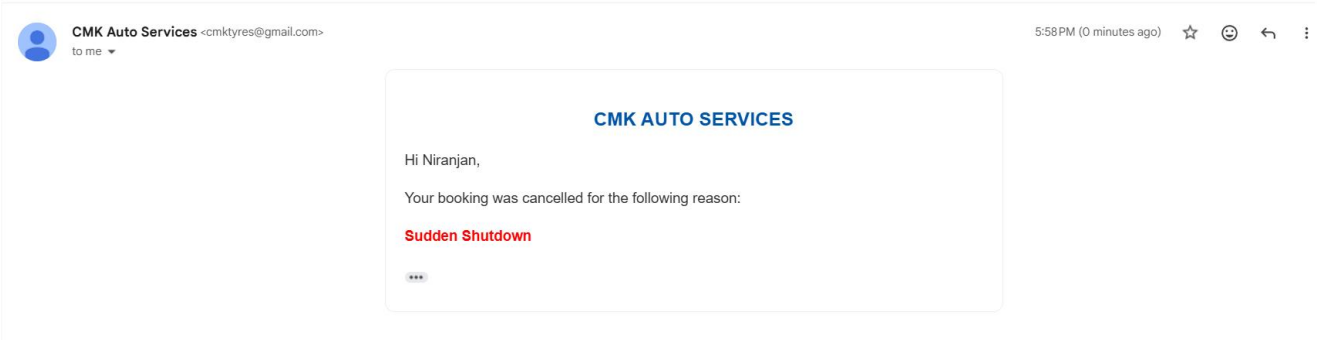

Figure 8.1.16

8.2 SAMPLE CODE

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';
import { generateInvoice } from '../utils/generateInvoice';

const AdminHistory = () => {
  const [orders, setOrders] = useState([]);
  const [searchTerm, setSearchTerm] = useState('');
  const [filterDate, setFilterDate] = useState('');
  const [loading, setLoading] = useState(true);

  const fetchHistory = async () => {
    try {
      setLoading(true);
      // Added a timestamp to the URL to prevent browser caching of old data
      const response = await axios.get(`http://localhost:5000/api/admin-
orders?t=${Date.now()}`);

      // 1. Only show Completed or Cancelled
      // 2. FORCED SORT: (b.id - a.id) ensures the newest IDs are always at the top
      const historyOnly = response.data
        .filter(o => o.status === 'Completed' || o.status === 'Cancelled')
        .sort((a, b) => b.id - a.id);

      setOrders(historyOnly);
    } catch (error) {
      console.error("History Fetch Error:", error);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => { fetchHistory(); }, []);
```

```
  const displayedOrders = orders.filter(o => {
    const searchString = searchTerm.toLowerCase();
    const matchesSearch =
      (o.id && o.id.toString().includes(searchString)) ||
      (o.customer_name && o.customer_name.toLowerCase().includes(searchString));

    // IMPROVED DATE MATCH: Handles both service_date and created_at formats
    const matchesDate = filterDate === '' ||
      (o.service_date && o.service_date.startsWith(filterDate)) ||
      (o.created_at && o.created_at.startsWith(filterDate));

    return matchesSearch && matchesDate;
  });

  if (loading) return <div className="pt-40 text-center font-black text-[#0054a6] animate-pulse
uppercase tracking-widest">Accessing Archives...</div>;

  return (
    <div className="pt-32 pb-12 px-4 sm:px-[5vw] md:px-[7vw] lg:px-[9vw] bg-gray-50 min-
h-screen">
      <div className="flex flex-col md:flex-row justify-between items-end mb-10 gap-6">
        <div>
          <h1 className="text-4xl font-black text-gray-900 uppercase tracking-tighter italic
leading-none">Job <span className="text-[#0054a6]">History</span></h1>
          <p className="text-gray-400 font-bold mt-2 uppercase text-[10px] tracking-
widest">Total Archived Records: {orders.length}</p>
        </div>

        <div className="flex gap-4 w-full md:w-auto">
          <input
            type="text"
            placeholder="Search Name or ID..."
            className="bg-white border border-gray-100 px-6 py-4 rounded-2xl text-sm font-bold
shadow-sm focus:border-[#0054a6] outline-none w-full md:w-64 transition-all focus:ring-2
focus:ring-blue-100"
            value={searchTerm}
            onChange={(e) => setSearchTerm(e.target.value)}
          />
          <input
```

```
            type="date"
            className="bg-white border border-gray-100 px-6 py-4 rounded-2xl text-sm font-bold shadow-sm focus:border-[#0054a6] outline-none transition-all focus:ring-2 focus:ring-blue-100"
            value={filterDate}
            onChange={(e) => setFilterDate(e.target.value)}
          />
        </div>
      </div>

      <div className="grid grid-cols-1 gap-4">
        {displayedOrders.length > 0 ? displayedOrders.map((order) => {
          // Safety parsing for services JSON
          let services = [];
          try {
            services = typeof order.services === 'string' ? JSON.parse(order.services) : order.services || [];
          } catch (e) { services = []; }

          return (
            <div key={order.id} className="bg-white p-8 rounded-[2.5rem] border border-gray-100 shadow-sm hover:shadow-md transition-all">
              <div className="flex flex-col lg:flex-row justify-between gap-8">
                <div className="flex-1">
                  <div className="flex items-center gap-4 mb-4">
                    <span className="text-[10px] font-black uppercase text-gray-300">#CMK-{order.id}</span>
                    <span className={`px-4 py-1 rounded-full text-[9px] font-black uppercase tracking-widest border ${
                      order.status === 'Completed' ? 'bg-green-50 text-green-600 border-green-100' : 'bg-red-50 text-red-600 border-red-100'
                    }`}>{order.status}</span>
                    <span className="text-[10px] font-black text-gray-400 uppercase">
                      Job Date: {order.service_date ? order.service_date.split('T')[0] : order.created_at?.split('T')[0]}
                    </span>
                  </div>

                  <h3 className="text-xl font-black text-gray-900 uppercase mb-4 tracking-
```

```
tight">{order.customer_name}</h3>

            <div className="flex flex-wrap gap-2 mb-6">
              {services.map((s, i) => (
                <span key={i} className="bg-gray-50 px-3 py-1 rounded-lg text-[10px] font-bold text-gray-500 border border-gray-100 uppercase">
                  {s.service || s.title}
                </span>
              ))}
            </div>

            <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
              <div>
                <p className="text-[9px] font-black text-gray-400 uppercase tracking-widest mb-1">User's Issue</p>
                <p className="text-xs font-bold text-gray-600 italic bg-gray-50 p-3 rounded-xl border border-gray-100 inline-block min-w-full">
                  "{order.problem_description || "No description provided"}"
                </p>
              </div>

              {order.status === 'Cancelled' && order.notes && (
                <div>
                  <p className="text-[9px] font-black text-red-500 uppercase tracking-widest mb-1">Workshop Note (Cancel Reason)</p>
                  <p className="text-xs text-red-700 font-bold italic bg-red-50 p-3 rounded-xl border border-red-100 inline-block min-w-full">
                    "{order.notes}"
                  </p>
                </div>
              )}
            </div>
          </div>

          <div className="flex flex-row lg:flex-col justify-between items-center lg:items-end lg:justify-center gap-4 border-t lg:border-t-0 lg:border-l border-gray-100 pt-6 lg:pt-0 lg:pl-10">
            <div className="text-right">
              <p className="text-[10px] font-black text-gray-400 uppercase tracking-widest
```

```
mb-1">Total Amount</p>
                <p className="text-3xl font-black text-gray-900 leading-
none">₹{order.total_price}</p>
              </div>

              {order.status === 'Completed' && (
                <button
                  onClick={() => generateInvoice(order)}
                  className="bg-black text-white px-6 py-3 rounded-xl text-[10px] font-black
uppercase tracking-widest hover:bg-[#0054a6] transition-all shadow-lg active:scale-95"
                >
                  Invoice ⬇
                </button>
              )}
            </div>
          </div>
        </div>
      );
    }) : (
      <div className="text-center py-20 bg-white rounded-[2.5rem] border border-dashed
border-gray-200 text-gray-400 font-bold uppercase tracking-widest">
        No archived records found matching criteria
      </div>
    )}
  </div>
 </div>
);
};

export default AdminHistory;


import React, { useEffect, useState } from 'react';
import axios from 'axios';
import { generateInvoice } from '../utils/generateInvoice';

const Orders = () => {
  const [orders, setOrders] = useState([]);
```

```
  const [loading, setLoading] = useState(true);
  const user = JSON.parse(localStorage.getItem('user'));

  const fetchOrders = async () => {
    if (!user) return;
    try {
      setLoading(true);
      // t=${Date.now()} prevents the browser from showing old cached data
      const response = await axios.get(`http://localhost:5000/api/user-
orders/${user.id}?t=${Date.now()}`);

      // FORCED SORT: Ensure highest ID is at the top
      const sortedData = response.data.sort((a, b) => b.id - a.id);

      setOrders(sortedData);
    } catch (error) {
      console.error("Fetch Error:", error);
    } finally {
      setLoading(false);
    }
  };

  const handleCancelOrder = async (orderId) => {
    if (!window.confirm("Are you sure you want to cancel this booking request?")) return;
    try {
      await axios.patch(`http://localhost:5000/api/cancel-order/${orderId}`);
      alert("Booking Cancelled.");
      fetchOrders();
    } catch (error) {
      alert("Failed to cancel order.");
    }
  };

  useEffect(() => { fetchOrders(); }, []);

  if (loading) return <div className="pt-40 text-center font-black text-gray-400 animate-pulse
uppercase tracking-widest">Loading Your Garage...</div>;
```

```
 return (
   <div className="pt-32 pb-12 px-4 sm:px-[5vw] md:px-[7vw] lg:px-[9vw] bg-gray-50 min-h-screen">
     <div className="max-w-6xl mx-auto">
       <h1 className="text-4xl font-black text-gray-900 mb-10 uppercase tracking-tighter italic">Service <span className="text-[#0054a6]">History</span></h1>

       <div className="space-y-6">
         {orders.length > 0 ? orders.map((order) => {
           const services = JSON.parse(order.services || '[]');

           return (
             <div key={order.id} className="bg-white p-8 rounded-[2.5rem] border border-gray-100 shadow-sm hover:shadow-md transition-all">
               <div className="flex flex-col md:flex-row justify-between gap-8">
                 <div className="flex-1">
                   <div className="flex items-center gap-3 mb-4">
                     <span className="bg-gray-100 text-gray-500 px-4 py-1 rounded-full text-[10px] font-black uppercase tracking-widest">Booking #{order.id}</span>
                     <span className={`px-4 py-1 rounded-full text-[10px] font-black uppercase tracking-widest border ${
                       order.status === 'Completed' ? 'bg-green-50 text-green-600 border-green-100' :
                       order.status === 'Cancelled' ? 'bg-red-50 text-red-600 border-red-100' :
                       'bg-orange-50 text-orange-600 border-orange-100'
                     }`}>
                       {order.status}
                     </span>
                   </div>

                   <h3 className="text-2xl font-black text-gray-900 uppercase mb-6 leading-none">
                     {services.map(s => s.title || s.service).join(" + ")}
                   </h3>

                   <div className="grid grid-cols-2 md:grid-cols-4 gap-6">
                     <div>
                       <p className="text-[10px] font-black text-gray-400 uppercase tracking-widest mb-1">Total Amount</p>
                       <p className="text-lg font-black text-gray-900">₹{order.total_price}</p>
                     </div>
```

```
                <div>
                  <p className="text-[10px] font-black text-gray-400 uppercase tracking-widest mb-1">Branch</p>
                  <p className="text-lg font-black text-gray-900">{order.branch}</p>
                </div>
                {order.service_date && (
                  <div>
                    <p className="text-[10px] font-black text-gray-400 uppercase tracking-widest mb-1">Service Date</p>
                    <p className="text-lg font-black text-gray-900">{new Date(order.service_date).toLocaleDateString()}</p>
                  </div>
                )}
              </div>
            </div>

            <div className="flex flex-col justify-center gap-3">
              {order.status === 'Completed' && (
                <button onClick={() => generateInvoice(order)} className="bg-gray-900 text-white px-8 py-4 rounded-2xl text-xs font-black uppercase hover:bg-[#0054a6] transition-all">Invoice ⬇</button>
              )}
              {order.status === 'Pending' && (
                <button onClick={() => handleCancelOrder(order.id)} className="bg-white text-red-600 border border-red-200 px-8 py-4 rounded-2xl text-xs font-black uppercase hover:bg-red-600 hover:text-white transition-all shadow-sm">Cancel Request</button>
              )}
            </div>
          </div>
        </div>
      );
    }) : (
      <div className="text-center py-20 bg-white rounded-[2.5rem] border border-dashed text-gray-300 font-black uppercase italic">Empty Garage</div>
    )}
  </div>
</div>
</div>
);
```

```
};

export default Orders;


import React, { useEffect, useState } from 'react';
import axios from 'axios';

const AdminOrders = () => {
  const [orders, setOrders] = useState([]);
  const [loading, setLoading] = useState(true);

  const fetchAllOrders = async (isInitial = false) => {
    try {
      if (isInitial) setLoading(true);
      const response = await axios.get('http://localhost:5000/api/admin-orders');
      setOrders(response.data);
    } catch (error) {
      console.error("Fetch Error:", error);
    } finally {
      if (isInitial) setLoading(false);
    }
  };

  const handleAcceptOrder = async (orderId, chosenDate) => {
    try {
      await axios.patch(`http://localhost:5000/api/update-status/${orderId}`, {
        status: 'Accepted',
        confirmed_date: chosenDate
      });
      fetchAllOrders(false);
    } catch (error) {
      alert("Error accepting order");
    }
  };

  const handleStatusChange = async (orderId, newStatus) => {
    try {
```

```
      await axios.patch(`http://localhost:5000/api/update-status/${orderId}`, { status: newStatus
});
      fetchAllOrders(false);
    } catch (error) {
      alert("Error updating status");
    }
  };

  const handleAdminCancel = async (orderId) => {
    const reason = window.prompt("Enter reason for cancellation:");
    if (reason === null) return;
    if (!reason.trim()) return alert("Reason is required.");

    try {
      await axios.patch(`http://localhost:5000/api/admin-cancel/${orderId}`, { reason });
      fetchAllOrders(false);
    } catch (error) {
      alert("Error cancelling order");
    }
  };

  useEffect(() => {
    fetchAllOrders(true);
  }, []);

  if (loading) return (
    <div className="pt-40 text-center font-black text-[#0054a6] animate-pulse">
      LOADING LIVE BOOKINGS...
    </div>
  );

  return (
    <div className="pt-32 pb-12 px-4 sm:px-[5vw] md:px-[7vw] lg:px-[9vw] bg-gray-50 min-
h-screen">

      <div className="mb-10">
        <h1 className="text-4xl font-black text-gray-900 uppercase tracking-tighter italic">
          Incoming <span className="text-[#0054a6]">Requests</span>
```

```
      </h1>
      <p className="text-gray-500 font-bold uppercase text-xs tracking-widest mt-2">
        Review and manage active bookings
      </p>
    </div>

    <div className="grid grid-cols-1 gap-6">
      {/* Filter out final states to keep the 'Requests' view clean */}
      {orders.filter(o => o.status !== 'Completed' && o.status !== 'Cancelled').map((order) => {

        // Safe Parsing for JSON fields
        let preferredDates = [];
        try {
          preferredDates = typeof order.preferred_dates === 'string'
            ? JSON.parse(order.preferred_dates)
            : order.preferred_dates || [];
        } catch (e) { preferredDates = []; }

        let services = [];
        try {
          services = typeof order.services === 'string'
            ? JSON.parse(order.services)
            : order.services || [];
        } catch (e) { services = []; }

        return (
          <div key={order.id} className="bg-white p-8 rounded-[2.5rem] border border-gray-
100 shadow-sm hover:shadow-md transition-all">
            <div className="flex flex-col lg:flex-row justify-between gap-8">

              <div className="flex-1">
                <div className="flex items-center gap-3 mb-4">
                  <span className="bg-blue-50 text-[#0054a6] px-4 py-1 rounded-full text-[10px]
font-black uppercase tracking-widest border border-blue-100">#{order.id}</span>
                  <span className={`px-4 py-1 rounded-full text-[10px] font-black uppercase
tracking-widest border ${
                    order.status === 'Pending' ? 'bg-orange-50 text-orange-600 border-orange-100' :
'bg-blue-50 text-blue-600 border-blue-100'
```

```
            }`}>
              {order.status}
            </span>
          </div>

          <h3 className="text-2xl font-black text-gray-900 uppercase leading-none mb-
1">{order.customer_name}</h3>
          <p className="text-gray-400 font-bold text-sm mb-6">{order.phone} |
{order.branch}</p>

          <div className="mb-6">
            <p className="text-[10px] font-black text-gray-400 uppercase tracking-widest
mb-1">Issue Reported</p>
            <p className="text-sm font-bold text-red-600 bg-red-50 p-4 rounded-2xl border
border-red-100 italic">
              "{order.problem_description || "No specific details provided."}"
            </p>
          </div>

          <div className="grid grid-cols-2 md:grid-cols-3 gap-4">
            {services.map((s, i) => (
              <div key={i} className="bg-gray-50 p-3 rounded-xl border border-gray-100">
                <p className="text-[9px] font-black text-gray-400 uppercase mb-
1">Service</p>
                <p className="text-xs font-bold text-gray-800 uppercase">{s.service || s.title ||
"Standard"}</p>
              </div>
            ))}
          </div>
        </div>

        <div className="lg:w-72 flex flex-col gap-4">
          {order.status === 'Pending' ? (
            <div className="bg-blue-50 p-6 rounded-3xl border border-blue-100">
              <p className="text-[10px] font-black text-[#0054a6] uppercase mb-4 text-center
tracking-widest">Select to Confirm</p>
              <div className="grid grid-cols-1 gap-2">
                {preferredDates.map((date, idx) => (
                  <button
```

```
                    key={idx}
                    onClick={() => handleAcceptOrder(order.id, date)}
                    className="bg-white border border-blue-200 py-3 rounded-xl text-xs font-
black hover:bg-[#0054a6] hover:text-white transition-all shadow-sm"
                  >
                    {date}
                  </button>
                ))}
              </div>
            </div>
          ) : (
            <div className="bg-gray-50 p-6 rounded-3xl text-center">
              <p className="text-[10px] font-black text-gray-400 uppercase mb-1">Confirmed
for:</p>
              <p className="text-2xl font-black text-gray-900">
                {order.service_date ? new Date(order.service_date).toLocaleDateString() :
'N/A'}
              </p>
              <button
                onClick={() => handleStatusChange(order.id, 'Completed')}
                className="mt-4 w-full bg-green-600 text-white py-4 rounded-2xl font-black
uppercase text-xs hover:bg-green-700 transition-colors"
              >
                Finish Job & Send Invoice
              </button>
            </div>
          )}
          <button
            onClick={() => handleAdminCancel(order.id)}
            className="text-red-400 text-[10px] font-black uppercase hover:text-red-600
transition-colors"
          >
            Cancel Booking
          </button>
        </div>

      </div>
    </div>
  );
```

```
        })}

        {/* Empty State */}
        {orders.filter(o => o.status !== 'Completed' && o.status !== 'Cancelled').length === 0 &&
(
          <div className="text-center py-20 bg-white rounded-[3rem] border-2 border-dashed
border-gray-200">
            <p className="text-gray-400 font-black uppercase italic text-xl">No active requests
right now.</p>
          </div>
        )}
      </div>
    </div>
  );
};

export default AdminOrders;


import React from 'react';
import { Link } from 'react-router-dom'; // Import Link
import frontImage from '../assets/front.png';

const HERO_CONTENT = {
  title: "Reliable Tyres & Wheel Alignment Services",
  description: "Building trust through quality service and long-term partnerships.",
  cta: "Explore Services",
};

const Home = () => {
  return (
    <section className="mt-12 grid grid-cols-1 md:grid-cols-2 gap-10 items-center px-4 md:px-
0">
      <div className="flex flex-col items-start text-left">
        <h1 className="text-4xl lg:text-5xl font-extrabold tracking-tight text-gray-900 mb-4">
          {HERO_CONTENT.title}
        </h1>
        <p className="text-lg text-gray-600 mb-8 max-w-md">
```

```
        {HERO_CONTENT.description}
      </p>

      {/* Wrap the button in a Link component pointing to /collection */}
      <Link to="/collection">
        <button
          className="bg-orange-500 text-white px-8 py-4 rounded-lg font-semibold hover:bg-
orange-600 focus:ring-4 focus:ring-orange-300 transition-all duration-200"
        >
          {HERO_CONTENT.cta}
        </button>
      </Link>
    </div>

    <div className="relative overflow-hidden rounded-xl shadow-2xl">
      <img
        src={frontImage}
        alt="Professional wheel alignment and tyre service"
        className="w-full h-auto object-cover transform hover:scale-105 transition-transform
duration-500"
        loading="eager"
        fetchpriority="high"
      />
    </div>
  </section>
 );
};

export default Home;
```

9. REFERENCES

BOOK REFERENCES

1. Learning React: Modern Patterns for Developing React Apps (2020). O'Reilly Media.
2. Node.js Design Patterns (2020). Packt Publishing.
3. Eloquent JavaScript: A Modern Introduction to Programming (2018). No Starch Press.
4. Web Development with Node and Express (2019). O'Reilly Media.
5. Learning MySQL (2017). O'Reilly Media.
6. Software Engineering: A Practitioner's Approach (2019). McGraw-Hill Education.

WEB REFERENCES

- React Official Documentation
- Node.js Official Documentation
- Express.js Documentation
- MySQL Official Documentation
- Nodemailer Documentation
- jsPDF Documentation
- Visual Studio Code Documentation